

Matplotlib — Giáo trình đầy đủ

Mục lục

Phần	Nội dung
Mở đầu	Mục tiêu, tiền đề, kiến trúc Figure/Axes, pyplot vs OOP, cài đặt
I	Biểu đồ cốt lõi, tùy chỉnh, subplot cơ bản, lưu file, Pandas
II	GridSpec, twin axes, layout, style, bài tập nâng cao
III	Lý thuyết & thực hành tốt (chọn biểu đồ, màu, EDA, ML)
IV	Bảng API + link docs
V	Phụ lục: thuật ngữ, checklist
VI	Bài tập tổng hợp: 5 bài có lời giải + 10 bài tự luyện

PHẦN I — Matplotlib cơ bản

Trực quan hóa dữ liệu với Matplotlib

Lesson Information | Thông tin bài học

Item	Description
Duration	90 minutes
Type	Lecture + Lab
Prerequisites	Lesson 4: Pandas Advanced

Learning Objectives | Mục tiêu bài học

#	Objective	Assessment
1	Create basic plots (line, bar, scatter, histogram)	Code exercises
2	Customize plot appearance	Lab work
3	Create subplots and complex figures	Code exercises
4	Save figures in various formats	Quiz

0. Nền tảng (đọc trước mục 1)

0.1 Mục tiêu Phần I (cụ thể hóa)

Sau Phần I, sinh viên **làm được**:

- Vẽ **line, scatter, bar, hist, pie, boxplot** với nhãn đầy đủ.
- Chỉnh **màu, marker, kích thước figure, legend, grid**.
- Tạo **lưới subplot** đơn giản và **chia sẻ trục**.
- **Lưu** hình PNG/PDF phù hợp báo cáo.
- Vẽ nhanh từ **pandas.DataFrame**.

0.2 Tiền đề

- Python, **NumPy** (mảng), **Pandas** (DataFrame) — xem Module 1–2.

0.3 Kiến trúc: Figure → Axes → (Artists)

Figure → chứa một hoặc nhiều Axes (vùng vẽ)
 Axes → trục tọa độ; gọi `.plot()`, `.scatter()`, `.set_xlabel()`, ...
 Artist → đường, chữ, hình – Matplotlib quản lý vẽ và xuất file.

- **Figure**: “khung” toàn bộ; `fig.savefig(...)`.
- **Axes**: một biểu đồ; khuyến nghị làm việc trực tiếp trên `ax`.

0.4 Hai API: `pyplot` so với hướng đối tượng

Tiêu chí	<code>plt.plot(...)</code>	<code>fig, ax = plt.subplots(); ax.plot(...)</code>
Phong cách	Stateful (bám “figure hiện tại”)	Rõ ràng, dễ tái sử dụng
Subplot nhiều panel	Dễ nhầm	Khuyến nghị cho bài tập & BTL
Notebook nhanh	Tiền 1–2 lệnh	Thêm 1 dòng tạo <code>fig, ax</code>

0.5 Cài đặt & Jupyter

```
pip install matplotlib
```

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
# Trong Jupyter – ảnh nhúng notebook:
# %matplotlib inline
```

1. Introduction to Matplotlib (10 min)

1.1 What is Matplotlib?

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python.

1.2 Installation and Import

```
pip install matplotlib
```

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# For Jupyter notebooks
%matplotlib inline
```

1.3 Two Interfaces

```
# Pyplot interface (simple, stateful)
plt.plot([1, 2, 3], [1, 4, 9])
plt.show()

# Object-oriented interface (more control)
fig, ax = plt.subplots()
ax.plot([1, 2, 3], [1, 4, 9])
plt.show()
```

2. Basic Plots (25 min)

2.1 Line Plot

```
# Simple line plot
x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.figure(figsize=(10, 6))
plt.plot(x, y)
plt.title('Sine Wave')
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.show()

# Multiple lines
y2 = np.cos(x)
plt.figure(figsize=(10, 6))
```

```
plt.plot(x, y, label='sin(x)')
plt.plot(x, y2, label='cos(x)')
plt.legend()
plt.title('Sine and Cosine')
plt.grid(True)
plt.show()

# Line styles
plt.plot(x, y, 'r-', linewidth=2)      # Red solid
plt.plot(x, y2, 'b--', linewidth=1.5) # Blue dashed
plt.plot(x, y + 1, 'g:', linewidth=2)  # Green dotted
plt.plot(x, y2 + 1, 'k-.', linewidth=1) # Black dash-dot
```

2.2 Scatter Plot

```
# Basic scatter
np.random.seed(42)
x = np.random.rand(50)
y = np.random.rand(50)

plt.figure(figsize=(8, 6))
plt.scatter(x, y)
plt.title('Scatter Plot')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()

# With size and color
sizes = np.random.rand(50) * 500
colors = np.random.rand(50)

plt.figure(figsize=(10, 6))
plt.scatter(x, y, s=sizes, c=colors, alpha=0.6, cmap='viridis')
plt.colorbar(label='Value')
plt.title('Scatter with Size and Color')
plt.show()
```

2.3 Bar Chart

```
# Simple bar chart
categories = ['A', 'B', 'C', 'D', 'E']
values = [23, 45, 56, 78, 32]

plt.figure(figsize=(8, 6))
plt.bar(categories, values, color='steelblue', edgecolor='black')
plt.title('Bar Chart')
plt.xlabel('Category')
plt.ylabel('Value')
plt.show()
```

```

# Horizontal bar
plt.figure(figsize=(8, 6))
plt.barh(categories, values, color='coral')
plt.title('Horizontal Bar Chart')
plt.show()

# Grouped bar chart
x = np.arange(len(categories))
width = 0.35
values1 = [23, 45, 56, 78, 32]
values2 = [30, 40, 50, 65, 45]

fig, ax = plt.subplots(figsize=(10, 6))
bars1 = ax.bar(x - width/2, values1, width, label='2023')
bars2 = ax.bar(x + width/2, values2, width, label='2024')
ax.set_xticks(x)
ax.set_xticklabels(categories)
ax.legend()
plt.title('Grouped Bar Chart')
plt.show()

# Stacked bar chart
fig, ax = plt.subplots(figsize=(10, 6))
ax.bar(categories, values1, label='2023')
ax.bar(categories, values2, bottom=values1, label='2024')
ax.legend()
plt.title('Stacked Bar Chart')
plt.show()

```

2.4 Histogram

```

# Basic histogram
data = np.random.randn(1000)

plt.figure(figsize=(10, 6))
plt.hist(data, bins=30, edgecolor='black')
plt.title('Histogram')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()

# With customization
plt.figure(figsize=(10, 6))
plt.hist(data, bins=30, density=True, alpha=0.7,
         color='steelblue', edgecolor='black')
plt.title('Normalized Histogram')
plt.show()

# Multiple histograms
data1 = np.random.randn(1000)

```

```
data2 = np.random.randn(1000) + 2

plt.figure(figsize=(10, 6))
plt.hist(data1, bins=30, alpha=0.5, label='Data 1')
plt.hist(data2, bins=30, alpha=0.5, label='Data 2')
plt.legend()
plt.show()
```

2.5 Pie Chart

```
sizes = [35, 25, 20, 15, 5]
labels = ['A', 'B', 'C', 'D', 'E']
explode = (0.1, 0, 0, 0, 0) # Explode first slice

plt.figure(figsize=(8, 8))
plt.pie(sizes, explode=explode, labels=labels, autopct='%1.1f%%',
        shadow=True, startangle=90)
plt.title('Pie Chart')
plt.axis('equal')
plt.show()
```

3. Plot Customization (20 min)

3.1 Colors and Styles

```
# Named colors
plt.plot(x, y, color='red')
plt.plot(x, y, color='steelblue')

# Hex colors
plt.plot(x, y, color='#FF5733')

# RGB tuple
plt.plot(x, y, color=(0.1, 0.2, 0.5))

# Line styles
plt.plot(x, y, linestyle='-')      # Solid
plt.plot(x, y, linestyle='--')    # Dashed
plt.plot(x, y, linestyle=':')     # Dotted
plt.plot(x, y, linestyle='-.')    # Dash-dot

# Markers
plt.plot(x, y, marker='o')        # Circle
plt.plot(x, y, marker='s')        # Square
plt.plot(x, y, marker='^')        # Triangle
plt.plot(x, y, marker='*')        # Star

# Combined format string
```

```
plt.plot(x, y, 'ro-')    # Red circles, solid line
plt.plot(x, y, 'bs--')   # Blue squares, dashed
```

3.2 Figure and Axes

```
# Figure size and DPI
fig = plt.figure(figsize=(12, 6), dpi=100)

# Title and labels
plt.title('Main Title', fontsize=16, fontweight='bold')
plt.xlabel('X Label', fontsize=12)
plt.ylabel('Y Label', fontsize=12)

# Axis limits
plt.xlim(0, 10)
plt.ylim(-1.5, 1.5)

# Axis ticks
plt.xticks([0, 2, 4, 6, 8, 10])
plt.yticks([-1, 0, 1], ['Low', 'Mid', 'High'])

# Grid
plt.grid(True)
plt.grid(True, linestyle='--', alpha=0.7)

# Legend
plt.legend(loc='upper right') # 'best', 'upper left', etc.
plt.legend(loc='upper right', fontsize=10, framealpha=0.9)
```

3.3 Text and Annotations

```
x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.figure(figsize=(10, 6))
plt.plot(x, y)

# Add text
plt.text(5, 0.5, 'Text here', fontsize=12, color='red')

# Add annotation with arrow
plt.annotate('Maximum', xy=(np.pi/2, 1), xytext=(3, 1.3),
            arrowprops=dict(arrowstyle='->', color='black'),
            fontsize=10)

# Add horizontal/vertical lines
plt.axhline(y=0, color='k', linestyle='--', alpha=0.5)
plt.axvline(x=np.pi, color='r', linestyle=':', alpha=0.5)
```

```
plt.show()
```

3.4 Styles and Themes

```
# Available styles
print(plt.style.available)

# Use a style
plt.style.use('seaborn-v0_8')
plt.style.use('ggplot')
plt.style.use('dark_background')
plt.style.use('fivethirtyeight')

# Reset to default
plt.style.use('default')

# Use temporarily
with plt.style.context('seaborn-v0_8'):
    plt.plot(x, y)
    plt.show()
```

4. Subplots (15 min)

4.1 Basic Subplots

```
# Create 2x2 grid of subplots
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Access individual subplots
axes[0, 0].plot(x, np.sin(x))
axes[0, 0].set_title('Sine')

axes[0, 1].plot(x, np.cos(x))
axes[0, 1].set_title('Cosine')

axes[1, 0].bar(['A', 'B', 'C'], [1, 2, 3])
axes[1, 0].set_title('Bar')

axes[1, 1].scatter(np.random.rand(50), np.random.rand(50))
axes[1, 1].set_title('Scatter')

plt.tight_layout()
plt.show()
```

4.2 Sharing Axes


```
# Share x-axis
fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True, figsize=(10, 8))
ax1.plot(x, np.sin(x))
ax2.plot(x, np.cos(x))
plt.show()

# Share y-axis
fig, (ax1, ax2) = plt.subplots(1, 2, sharey=True, figsize=(12, 5))
ax1.plot(x, y)
ax2.plot(x, y + 0.5)
plt.show()
```

4.3 Custom Layout

```
# GridSpec for complex layouts
from matplotlib.gridspec import GridSpec

fig = plt.figure(figsize=(12, 8))
gs = GridSpec(3, 3, figure=fig)

ax1 = fig.add_subplot(gs[0, :])      # Top row, all columns
ax2 = fig.add_subplot(gs[1, :-1])   # Middle row, first 2 cols
ax3 = fig.add_subplot(gs[1:, -1])    # Right column, last 2 rows
ax4 = fig.add_subplot(gs[-1, 0])     # Bottom left
ax5 = fig.add_subplot(gs[-1, 1])     # Bottom middle

ax1.plot(x, y)
ax1.set_title('Full Width')

ax2.bar(['A', 'B', 'C'], [1, 2, 3])
ax3.scatter(np.random.rand(20), np.random.rand(20))
ax4.hist(np.random.randn(100))
ax5.pie([1, 2, 3])

plt.tight_layout()
plt.show()
```

5. Saving Figures (10 min)

5.1 Save to File

```
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(x, y)

# Save as PNG
fig.savefig('figure.png')
fig.savefig('figure.png', dpi=300)
```

```
# Save as PDF
fig.savefig('figure.pdf')

# Save as SVG (vector)
fig.savefig('figure.svg')

# Common options
fig.savefig('figure.png',
            dpi=300,
            bbox_inches='tight',    # Remove extra whitespace
            facecolor='white',     # Background color
            transparent=False      # Transparent background
        )
```

5.2 Supported Formats

Format	Description
PNG	Raster, best for web
PDF	Vector, best for print
SVG	Vector, editable
EPS	Vector, publications
JPEG	Raster, photos

6. Plotting with Pandas (10 min)

```
# Direct plotting from DataFrame
df = pd.DataFrame({
    'Month': ['Jan', 'Feb', 'Mar', 'Apr', 'May'],
    'Sales': [100, 120, 150, 130, 170],
    'Expenses': [80, 90, 100, 95, 110]
})

# Line plot
df.plot(x='Month', y=['Sales', 'Expenses'], figsize=(10, 6))
plt.title('Sales vs Expenses')
plt.show()

# Bar plot
df.plot(x='Month', y='Sales', kind='bar', figsize=(10, 6))
plt.show()

# Multiple plot types
df.set_index('Month')[['Sales', 'Expenses']].plot(kind='bar')
plt.show()
```

```
# Histogram
df['Sales'].plot(kind='hist', bins=10)
plt.show()

# Scatter
df.plot(x='Sales', y='Expenses', kind='scatter', figsize=(8, 6))
plt.show()
```

Practice Exercises | Bài tập thực hành

Exercise 5.1: Basic Plots

```
# Create the following plots:
# 1. Line plot of  $y = x^2$  for  $x$  in  $[-5, 5]$ 
# 2. Scatter plot with 100 random points, color by quadrant
# 3. Bar chart of 5 products and their sales
# 4. Histogram of 1000 normally distributed values
```

Exercise 5.2: Customization

```
# Create a publication-quality figure:
# - Two subplots (side by side)
# - Left: line plot with grid, legend, labels
# - Right: bar chart with annotations
# - Add overall title
# - Save as PNG (300 DPI) and PDF
```

Exercise 5.3: Data Visualization

```
# Using Titanic dataset:
# Create a 2x2 subplot figure:
# 1. Bar chart of survival by class
# 2. Histogram of ages
# 3. Pie chart of gender distribution
# 4. Scatter plot of age vs fare
```

Summary | Tóm tắt

Common Plot Types

Function	Plot Type
<code>plt.plot()</code>	Line

Function	Plot Type
<code>plt.scatter()</code>	Scatter
<code>plt.bar()</code>	Bar
<code>plt.hist()</code>	Histogram
<code>plt.pie()</code>	Pie
<code>plt.boxplot()</code>	Box

Key Customizations

Function	Purpose
<code>plt.figure()</code>	Create figure
<code>plt.title()</code>	Add title
<code>plt.xlabel/ylabel()</code>	Axis labels
<code>plt.legend()</code>	Add legend
<code>plt.grid()</code>	Add grid
<code>plt.savefig()</code>	Save figure

PHẦN II — Matplotlib nâng cao

Matplotlib nâng cao: Subplots và Customization

Mục tiêu bài học

Sau bài học này, sinh viên có thể:

- Tạo nhiều biểu đồ trong một figure (subplots)
- Tùy chỉnh giao diện biểu đồ
- Lưu biểu đồ với chất lượng cao

1. Subplots

1.1. Tạo Subplots cơ bản

```
import matplotlib.pyplot as plt
import numpy as np

# Tạo figure với 2x2 subplots
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
```

```

# Data
x = np.linspace(0, 10, 100)

# Plot 1: Line plot
axes[0, 0].plot(x, np.sin(x), 'b-', label='sin(x)')
axes[0, 0].set_title('Line Plot')
axes[0, 0].legend()
axes[0, 0].grid(True)

# Plot 2: Scatter plot
axes[0, 1].scatter(np.random.rand(50), np.random.rand(50), c='red',
alpha=0.5)
axes[0, 1].set_title('Scatter Plot')

# Plot 3: Bar chart
categories = ['A', 'B', 'C', 'D']
values = [23, 45, 56, 78]
axes[1, 0].bar(categories, values, color='green')
axes[1, 0].set_title('Bar Chart')

# Plot 4: Histogram
data = np.random.randn(1000)
axes[1, 1].hist(data, bins=30, edgecolor='black')
axes[1, 1].set_title('Histogram')

plt.tight_layout()
plt.show()

```

1.2. Subplots với kích thước khác nhau

```

# GridSpec cho layout phức tạp
from matplotlib.gridspec import GridSpec

fig = plt.figure(figsize=(12, 8))
gs = GridSpec(2, 3, figure=fig)

# Subplot chiếm 2 cột
ax1 = fig.add_subplot(gs[0, :2])
ax1.plot(x, np.sin(x))
ax1.set_title('Wide Plot')

# Subplot bình thường
ax2 = fig.add_subplot(gs[0, 2])
ax2.bar(['A', 'B'], [10, 20])
ax2.set_title('Narrow Plot')

# Subplot chiếm toàn bộ hàng dưới
ax3 = fig.add_subplot(gs[1, :])
ax3.scatter(np.random.rand(100), np.random.rand(100))
ax3.set_title('Full Width Plot')

```

```
plt.tight_layout()
plt.show()
```

2. Customization

2.1. Colors và Markers

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y1, y2 = np.sin(x), np.cos(x)
y3, y4 = np.sin(x) * 0.5, np.cos(x) * 0.5

fig, ax = plt.subplots(figsize=(10, 6))

# Line styles
ax.plot(x, y1, 'r-', linewidth=2, label='Solid Red')
ax.plot(x, y2, 'b--', linewidth=1.5, label='Dashed Blue')
ax.plot(x, y3, 'g:', linewidth=2, label='Dotted Green')
ax.plot(x, y4, 'm-.', linewidth=1, label='Dash-dot Magenta')

# Markers (ví dụ trên đoạn ngắn)
ax.plot(x[:20], y1[:20], 'o-', markersize=8, markerfacecolor='yellow',
        markeredgecolor='black', markeredgewidth=2, label='With markers')

ax.legend(loc='upper right', fontsize=8)
ax.set_title('Colors, line styles, markers')
plt.show()

# Scatter với colormap (figure riêng để colorbar gọn)
fig, ax = plt.subplots(figsize=(8, 6))
sizes = np.random.rand(50) * 200 + 20
colors = np.random.rand(50)
sc = ax.scatter(np.random.rand(50) * 10, np.random.rand(50), c=colors,
               cmap='viridis', s=sizes, alpha=0.7)
plt.colorbar(sc, ax=ax, label='Value')
ax.set_title('Scatter with colormap')
plt.show()
```

2.2. Axes và Labels

```
fig, ax = plt.subplots(figsize=(10, 6))

ax.plot(x, y)

# Title và labels
ax.set_title('My Plot', fontsize=16, fontweight='bold')
```

```

ax.set_xlabel('X Axis', fontsize=12)
ax.set_ylabel('Y Axis', fontsize=12)

# Axis limits
ax.set_xlim(0, 10)
ax.set_ylim(-1, 1)

# Ticks
ax.set_xticks([0, 2, 4, 6, 8, 10])
ax.set_xticklabels(['Zero', 'Two', 'Four', 'Six', 'Eight', 'Ten'])

# Grid
ax.grid(True, linestyle='--', alpha=0.7)

# Legend
ax.legend(loc='upper right', fontsize=10)

```

2.3. Annotations

```

# Giả sử đã có fig, ax, x, y (mảng numpy)
x = np.linspace(0, 10, 100)
y = np.sin(x)
x_max, y_max = x[np.argmax(y)], y.max()

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(x, y)

# Text annotation
ax.annotate('Maximum', xy=(x_max, y_max), xytext=(x_max + 1, y_max + 0.3),
           arrowprops=dict(arrowstyle='->', color='red'),
           fontsize=12, color='red')

# Vertical/Horizontal lines
ax.axhline(y=0, color='k', linestyle='-', linewidth=0.5)
ax.axvline(x=5, color='r', linestyle='--', linewidth=1)

# Shaded region giữa hai đường
ax.fill_between(x, np.sin(x), np.cos(x), alpha=0.3, color='blue')
plt.show()

```

2.4. Trục Y kép (twin axes)

Khi cần vẽ **hai chuỗi cùng trục X** nhưng **hai đơn vị khác nhau** (vd: nhiệt độ °C và độ ẩm %). **Cảnh báo:** dễ gây hiểu nhầm — dùng khi thật sự cần và ghi rõ nhãn hai bên.

```

fig, ax1 = plt.subplots(figsize=(10, 6))
x = np.arange(1, 13)
temp = np.random.uniform(20, 35, 12)
rain = np.random.uniform(0, 300, 12)

```

```

ax1.bar(x, rain, alpha=0.4, color='steelblue', label='Rain (mm)')
ax1.set_xlabel('Month')
ax1.set_ylabel('Rainfall (mm)', color='steelblue')
ax1.tick_params(axis='y', labelcolor='steelblue')

ax2 = ax1.twinx()
ax2.plot(x, temp, color='orangered', marker='o', linewidth=2, label='Temp
(°C)')
ax2.set_ylabel('Temperature (°C)', color='orangered')
ax2.tick_params(axis='y', labelcolor='orangered')
plt.title('Twin axes example')
plt.show()

```

2.5. constrained_layout vs tight_layout

- **plt.tight_layout()**: điều chỉnh padding sau khi vẽ; nhanh, đôi khi overlap với title subtitle.
- **fig, ax = plt.subplots(..., constrained_layout=True)**: bố cục **ràng buộc** tự động, thường ổn định hơn với colorbar và nhiều subplot.

```

fig, axes = plt.subplots(2, 2, figsize=(10, 8), constrained_layout=True)
for ax in axes.flat:
    ax.plot([0, 1], [0, 1])
fig.suptitle('Using constrained_layout=True')
plt.show()

```

3. Lưu Figure

3.1. Các định dạng

```

# PNG (high quality)
plt.savefig('plot.png', dpi=300, bbox_inches='tight')

# PDF (vector, tốt cho publications)
plt.savefig('plot.pdf', bbox_inches='tight')

# SVG (vector, tốt cho web)
plt.savefig('plot.svg', bbox_inches='tight')

# JPEG
plt.savefig('plot.jpg', dpi=150, quality=95)

```

3.2. Transparent background

```

plt.savefig('plot.png', dpi=300, transparent=True, bbox_inches='tight')

```

4. Styles

4.1. Built-in styles

```
# Xem các styles có sẵn
print(plt.style.available)

# Sử dụng style
plt.style.use('seaborn-v0_8-whitegrid')
plt.style.use('ggplot')
plt.style.use('dark_background')

# Reset về default
plt.style.use('default')
```

4.2. Custom style

```
# Temporary style
with plt.style.context('seaborn-v0_8-darkgrid'):
    plt.plot(x, y)
plt.show()
```

5. Bài tập

1. Tạo figure với 2x2 subplots chứa 4 loại biểu đồ khác nhau
2. Customize colors, labels, và legends cho từng subplot
3. Lưu figure dạng PNG với dpi=300
4. Thử ít nhất 3 styles khác nhau và so sánh
5. Tạo một figure dùng **GridSpec** với ít nhất 3 vùng subplot kích thước khác nhau
6. (Tùy chọn) Vẽ **twin axes** với dữ liệu giả lập (hai đơn vị khác nhau)

PHẦN III — Lý thuyết & thực hành tốt (trực quan hóa dữ liệu)

Mục tiêu học tập

Sau bài này, sinh viên có thể:

Mục tiêu

- | | |
|---|--|
| 1 | Giải thích mục đích của trực quan hóa trong phân tích dữ liệu và ML (EDA, báo cáo, giao tiếp) |
|---|--|

#	Mục tiêu
2	Chọn loại biểu đồ phù hợp với kiểu dữ liệu (số, phân loại, thời gian) và câu hỏi phân tích
3	Áp dụng nguyên tắc thiết kế (rõ ràng, trung thực với dữ liệu, hạn chế gây hiểu nhầm)
4	Dùng màu sắc hợp lý (phân loại vs thứ tự vs diverging); nhận biết vấn đề mù màu
5	Lập checklist EDA bằng biểu đồ; kể chuyện với dữ liệu (title, caption, luận điểm)
6	Chuẩn bị hình xuất bản/in ấn (DPI, vector, font)

1. Vai trò của trực quan hóa dữ liệu

1.1. Ba mục đích chính

- Khám phá (Exploration – EDA)**
Nhìn nhanh phân phối, outlier, mối quan hệ, mất cân bằng lớp — trước khi mô hình hóa.
- Kiểm định & chẩn đoán (Diagnostics)**
Residual plot, learning curve, confusion matrix, feature importance — đánh giá mô hình.
- Giao tiếp (Communication)**
Báo cáo cho người không đọc code: thông điệp phải **đọc được từ hình**, không chỉ từ bảng số.

1.2. Giới hạn của “chỉ nhìn bảng”

- Bảng `describe()` không cho thấy **đa đỉnh** (bimodal), **outlier cực đoan**, **quan hệ phi tuyến**.
- Correlation Pearson chỉ đo **tuyến tính**; scatter plot mới gợi ý dạng cong hoặc cụm.

2. Nguyên tắc thiết kế (tóm tắt có học thuật)

Các tác giả kinh điển (Edward Tufte, Stephen Few, ...) nhấn mạnh các ý sau — áp dụng được ngay với Matplotlib/Seaborn:

Nguyên tắc	Ý nghĩa thực hành
Data-ink ratio	Ưu tiên mực in cho dữ liệu ; giảm grid dày, khung thừa, hiệu ứng trang trí không mang thông tin.
Chartjunk	Tránh 3D giả tạo, hình nền rối, gradient chỉ để “đẹp” nhưng che pattern.
Integrity	Trục và scale phải trung thực — không cắt trục Y để phóng đại chênh lệch nhỏ (xem mục 6).
Small multiples	Nhiều panel cùng scale (FacetGrid, subplots) so sánh nhóm tốt hơn một hình quá tải.
Label đầy đủ	Title mô tả nội dung , không chỉ “Biểu đồ 1”. Ghi đơn vị (USD, %, năm).

3. Chọn loại biểu đồ theo câu hỏi phân tích

3.1. Ma trận quyết định nhanh

Câu hỏi	Kiểu biến	Gợi ý biểu đồ (ưu tiên)
Phân phối một biến số?	1 biến liên tục	Histogram, KDE, ECDF
So sánh phân phối theo nhóm?	Số + phân loại	Box, Violin, Strip/Swarm + Box
Tỷ lệ / số đếm theo nhóm?	Phân loại	Bar (count), horizontal bar nếu nhãn dài
Xu hướng theo thời gian?	Thời gian + số	Line plot (Matplotlib/Seaborn lineplot)
Quan hệ hai biến số?	2 biến số	Scatter, hexbin (đông điểm), jointplot
Nhiều biến số cùng lúc (EDA)?	Nhiều cột số	Pairplot, scatter matrix, correlation heatmap
Tương quan giữa nhiều biến số?	Ma trận số	Heatmap correlation, clustermap
Phần trăm thành phần ($\leq 5-7$ mục)?	Thành phần	Bar tích lũy 100%, hoặc pie có nhãn % (pie dễ đọc sai — dùng cẩn thận)

3.2. Phân loại theo “grammar of graphics” (khái niệm)

- **Distribution** (phân phối): hist, kde, rug, box, violin.
- **Relationship** (quan hệ): scatter, line, regplot.
- **Comparison** (so sánh): bar, pointplot.
- **Composition** (cấu thành): stacked bar, treemap (ngoài scope cơ bản).

Seaborn tổ chức API theo hướng này: **relplot**, **displot**, **catplot**.

4. Box plot & Violin plot — đọc đúng thống kê

4.1. Box plot (tùy implementation)

Thông thường:

- **Hộp**: từ Q1 đến Q3 (khoảng tứ phân vị — IQR).
- **Đường giữa hộp**: median (Q2).
- **Râu (whiskers)**: thường tới giá trị xa nhất trong khoảng $(Q1 - 1.5 \times IQR)$ đến $(Q3 + 1.5 \times IQR)$ (tùy thư viện).
- **Điểm ngoài râu**: thường được đánh dấu là **outlier** (gợi ý kiểm tra, không tự động “xóa”).

4.2. Violin plot

Kết hợp **mirror KDE** hai bên — thấy được **đa đỉnh** mà box plot che giấu. Phù hợp khi phân phối không đối xứng hoặc có nhiều cụm.

4.3. Kết hợp Strip/Swarm + Box

Xem **từng điểm** (strip/swarm) tránh hiểu nhầm khi sample nhỏ hoặc khi box “che” hết biến thiên.

5. Màu sắc & cảm nhận thị giác

5.1. Ba loại palette (Color Brewer / chuẩn khoa học dữ liệu)

Loại	Khi nào dùng	Ví dụ Seaborn / Matplotlib
Sequential	Dữ liệu thứ tự một chiều (thấp → cao)	viridis , Blues , YlOrRd
Diverging	Dữ liệu có điểm giữa có ý nghĩa (0, trung bình)	coolwarm , RdBu_r
Qualitative (categorical)	Nhóm không thứ tự	Set2 , tab10 , husl

Lưu ý: Dùng **cùng một palette sequential** cho heatmap correlation có **center=0** (diverging) để dương/âm dễ phân biệt.

5.2. mù màu (color blindness)

- Tránh **đỏ–xanh** làm hai nhóm duy nhất phân biệt (khó với deuteranopia).
- Dùng **pattern/hatch** hoặc **nhãn trực tiếp** bổ sung.
- Kiểm tra bằng công cụ simulate (browser extension) trước khi báo cáo quan trọng.

5.3. Không dùng rainbow cho dữ liệu liên tục

Rainbow **không đơn điệu** về độ sáng — não người đọc sai thứ tự. Ưu tiên **viridis**, **cividis**, **plasma**.

6. Trục, thang đo & các lỗi gây hiểu nhầm thường gặp

6.1. Trục Y không bắt đầu từ 0 (bar chart)

Với **bar chart**, cắt đáy trục Y làm **chênh lệch nhỏ trông lớn** — dễ bị coi là thiếu trung thực. Nếu cần nhấn mạnh biến động nhỏ trên baseline cao, dùng **line chart** hoặc ghi rõ **break** trên trục.

6.2. Histogram — số bins

Bins quá ít → mất chi tiết; quá nhiều → nhiễu. Có thể thử **bins='auto'**, **fd**, hoặc so sánh 2–3 giá trị bins trong cùng báo cáo.

6.3. Trục log

Dùng **log scale** khi dữ liệu **span nhiều độ lớn** (population, giá cổ phiếu, frequency) — ghi rõ trên nhãn trục.

6.4. Pie chart

Chỉ hợp **ft slice ($\leq 5-7$)**, tỷ lệ khác biệt rõ. Nhiều slice nhỏ → **bar chart** xếp theo giá trị dễ đọc hơn.

6.5. Trục kép (dual axis)

Hai thang đo khác đơn vị trên cùng hình — dễ **so sánh sai** hai chuỗi. Chỉ dùng khi thật sự cần và **giải thích rõ** trong caption.

7. Quy trình EDA với visualization — checklist

Áp dụng cho mỗi dataset mới (notebook EDA):

- ☐ **Shape & dtypes:** `info()`, `describe()` — sau đó **hist/KDE** cho mọi cột số.
- ☐ **Missing:** heatmap missing (optional) hoặc bar count missing theo cột.
- ☐ **Target:** nếu supervised — phân phối target (bar cho class, hist cho regression).
- ☐ **Categorical vs target:** countplot, barplot có estimator=mean/median.
- ☐ **Correlation:** heatmap (chỉ số số); cẩn thận **không kết luận nhân quả**.
- ☐ **Outlier:** boxplot hoặc scatter với cột nghi ngờ.
- ☐ **Bivariate:** scatter / pairplot cho các cặp quan trọng.
- ☐ **Theo nhóm:** FacetGrid hoặc **hue** để thấy tương tác nhóm.

Hướng dẫn thực hành chi tiết (**code + dataset Titanic**, mở rộng **Diamonds/Airbnb**):

[Guide_EDA_Visualization_Titanic.md](#).

8. Kể chuyện với dữ liệu (Storytelling)

1. **Một hình — một thông điệp** chính.
2. **Title** nêu **kết luận nhẹ** (vd: "Doanh thu Q4 tăng chủ yếu ở khu vực B") thay vì "Biểu đồ doanh thu".
3. **Caption** ghi nguồn dữ liệu, đơn vị, thời điểm snapshot.
4. Thứ tự trình bày: **context** → **hình chính** → **kết luận** → (optional) **hình phụ** hỗ trợ.

9. Figure-level vs Axes-level trong Seaborn

Loại	Hàm ví dụ	Đặc điểm
Axes-level	<code>sns.scatterplot(ax=...)</code>	Vẽ lên Axes có sẵn; dễ tích hợp subplot Matplotlib.
Figure-level	<code>sns.relplot</code> , <code>sns.catplot</code> , <code>sns.displot</code>	Tạo FacetGrid / FacetGrid -like; dùng col , row , hue thống nhất.

Quy tắc: Dashboard nhiều panel tùy ý → thường **Axes-level + plt.subplots**. So sánh nhóm theo lưới chuẩn → **relplot/catplot/displot**.

10. Xuất bản & in ấn

Yêu cầu	Gợi ý
---------	-------

Yêu cầu	Gợi ý
Bài báo / slide HD	<code>dpi=300</code> (raster), hoặc PDF/SVG (vector).
Kích thước	<code>figsize + bbox_inches='tight'</code> khi save.
Font	<code>plt.rcParams['font.size']</code> — đủ lớn cho slide (thường ≥ 14 –16 cho nhìn trực khi trình chiếu).
Nền trong suốt	<code>transparent=True</code> cho overlay slide (tùy nền).

11. Liên hệ với Machine Learning

Giai đoạn ML	Biểu đồ gợi ý
EDA	Correlation heatmap, pairplot, phân phối target, missing pattern
Preprocessing sau	So sánh phân phối trước/sau scale (optional)
Training	Learning curve, validation score theo epoch (deep learning — ngoài sklearn cơ bản)
Classification	Confusion matrix, ROC, Precision-Recall (imbalanced)
Regression	Residual plot, Actual vs Predicted
Clustering	Silhouette visualization, scatter 2D/PCA của cluster

12. Tài liệu tham khảo

1. Tufte, E. — *The Visual Display of Quantitative Information*
2. Few, S. — *Show Me the Numbers*
3. Wilkinson, L. — *The Grammar of Graphics* (khái niệm lớp biểu đồ)
4. Matplotlib Gallery: <https://matplotlib.org/stable/gallery/>
5. Seaborn Tutorial: <https://seaborn.pydata.org/tutorial.html>
6. Color Brewer: <https://colorbrewer2.org/>

13. Bài tập tự luận (không code)

1. Cho bài toán “so sánh lương theo phòng ban” (1 biến số, 1 categorical), bạn chọn box hay violin hay cả hai? Vì sao?
2. Nêu một tình huống **pie chart** chấp nhận được và một tình huống nên thay bằng **bar chart**.
3. Giải thích tại sao correlation heatmap **không chứng minh nhân quả**.

PHẦN IV — Tham chiếu API (Matplotlib)

Giới thiệu

Phần này là **bảng tra cứu nhanh**; mọi tham số đầy đủ xem [Pyplot](#) và [Axes API](#).

Matplotlib — Tài liệu chính thức

Nội dung	URL
Pyplot (state machine)	https://matplotlib.org/stable/api/pyplot_summary.html
Axes (OOP)	https://matplotlib.org/stable/api/axes_api.html
Figure	https://matplotlib.org/stable/api/figure_api.html
Gallery	https://matplotlib.org/stable/gallery/index.html

Matplotlib `pyplot` / `Axes` — API thường dùng

Figure & subplot

Hàm / phương thức	Ý nghĩa	Tham số ghi nhớ
<code>plt.figure(figsize=(w,h), dpi=...)</code>	Tạo figure mới	<code>figsize</code> inch, <code>dpi</code>
<code>plt.subplots(nrows, ncols, ...)</code>	Figure + mảng Axes	<code>sharex</code> , <code>sharey</code> , <code>constrained_layout</code>
<code>fig.add_subplot()</code>	Thêm subplot	<code>projection</code> (polar, 3d)
<code>plt.subplot_mosaic()</code>	Layout theo tên ô	Matplotlib ≥ 3.3
<code>GridSpec</code>	Lưới không đều	<code>width_ratios</code> , <code>height_ratios</code>

Vẽ cơ bản (trên `ax` hoặc `plt`)

API	Mô tả	Tham số thường gặp
<code>plot(x, y, ...)</code>	Đường / marker	<code>color</code> , <code>linestyle</code> , <code>linewidth</code> , <code>marker</code> , <code>label</code>
<code>scatter(x, y, ...)</code>	Điểm	<code>s</code> (size), <code>c</code> (màu/số), <code>cmap</code> , <code>alpha</code>
<code>bar(x, height, ...)</code>	Cột dọc	<code>width</code> , <code>color</code> , <code>edgecolor</code> , <code>tick_label</code>
<code>barh(y, width, ...)</code>	Cột ngang	Tương tự <code>bar</code>
<code>hist(x, bins=..., ...)</code>	Histogram	<code>bins</code> , <code>density</code> , <code>alpha</code> , <code>edgecolor</code>
<code>pie(x, ...)</code>	Tròn	<code>labels</code> , <code>autopct</code> , <code>explode</code>
<code>boxplot(data, ...)</code>	Hộp	<code>labels</code> , <code>vert</code>

API	Mô tả	Tham số thường gặp
<code>fill_between(x, y1, y2, ...)</code>	Vùng tô	<code>alpha</code> , <code>color</code>
<code>imshow(arr, ...)</code>	Ảnh / ma trận 2D	<code>cmap</code> , <code>aspect</code>

Trục & chữ

API	Ý nghĩa
<code>set_xlim</code> , <code>set_ylim</code>	Giới hạn trục
<code>set_xticks</code> , <code>set_xticklabels</code>	Tick & nhãn
<code>set_xlabel</code> , <code>set_ylabel</code> , <code>set_title</code>	Nhãn
<code>legend(loc=..., ...)</code>	Chú thích
<code>grid(True, ...)</code>	Lưới
<code>tick_params</code>	Cỡ chữ tick, màu
<code>set_scale('log')</code>	Trục log

Lưu file

API	Ghi chú
<code>savefig(fname, dpi=300, bbox_inches='tight', ...)</code>	Gọi trước <code>plt.close()</code> nếu batch
Định dạng	<code>.png</code> , <code>.pdf</code> , <code>.svg</code> , <code>.jpg</code> — PDF/SVG vector

Vẽ bổ sung (Axes / pyplot)

API	Mô tả	Ghi chú
<code>errorbar(x, y, yerr=..., xerr=...)</code>	Điểm + thanh sai số	<code>fmt</code> , <code>capsize</code> , <code>ecolor</code>
<code>hexbin(x, y, gridsize=...)</code>	Hexagon mật độ 2D	Dữ liệu rất đông điểm
<code>stackplot(x, y1, y2, ...)</code>	Diện tích chồng	Chuỗi thời gian / thành phần
<code>step(x, y, where=...)</code>	Bậc thang	<code>where='mid' 'post' 'pre'</code>
<code>axvline</code> / <code>axhline</code>	Đường dọc/ngang	Mốc tham chiếu
<code>text</code> , <code>figtext</code>	Chữ tự do	Tọa độ data hoặc figure

`matplotlib.rcParams` (một số khóa hay chỉnh)

Khóa	Ý nghĩa
------	---------

Khóa	Ý nghĩa
<code>figure(figsize</code>	Mặc định (6.4, 4.8) inch
<code>figure.dpi</code>	Độ phân giải figure
<code>font.size</code>	Cỡ chữ cơ bản
<code>axes.grid</code>	Bật lưới mặc định <code>True/False</code>
<code>axes.prop_cycle</code>	Chu kỳ màu đường (mặc định)

```
import matplotlib as mpl
mpl.rcParams['figure.figsize'] = (8, 5)
mpl.rcParams['font.size'] = 11
```

Bộ nhớ & backend (Jupyter / script)

- `plt.close()` hoặc `plt.close('all')` sau nhiều `savefig` để giải phóng RAM.
- Jupyter: `%matplotlib inline` (ảnh tĩnh); `%matplotlib widget` cần `ipyml` cho tương tác.

OOP vs pyplot

Khuyến nghị: `fig, ax = plt.subplots(); ax.plot(...); ax.set_title(...)`
 pyplot: `plt.plot(...)` – nhanh nhưng stateful, dễ lẫn khi nhiều figure.

Phần API — Ghi chú

- Phần Seaborn của bảng API nằm trong [Seaborn_Complete.md](#) (Phần IV).

PHẦN V — Phụ lục

V.1 Thuật ngữ (chọn lọc)

Thuật ngữ	Giải thích ngắn
Figure	Toàn bộ "trang" vẽ
Axes	Vùng có trục X/Y (một plot)
dpi	Điểm/inch — độ nét khi raster (PNG)
cmap	Bảng màu (colormap)
tight_layout	Tự chỉnh khoảng trắng giữa các subplot

Thuật ngữ**Giải thích ngắn****vector (PDF/SVG)** Phóng to không vỡ nét như raster**V.2 Checklist trước khi nộp hình (báo cáo / BTL)**

- ☐ Trục có **đơn vị** và **tiêu đề** mô tả nội dung (không chỉ "Biểu đồ 1").
- ☐ Chữ đủ lớn khi in hoặc chiếu slide.
- ☐ Bar chart: cân nhắc **gốc trục 0** (xem Phần III).
- ☐ File nộp: **PDF hoặc PNG ≥ 300 dpi** cho hình quan trọng.
- ☐ Đã `plt.tight_layout()` hoặc `constrained_layout=True` nếu nhiều panel.

PHẦN VI — Bài tập tổng hợp (có lời giải + tự luyện)

Cách dùng: Làm **VI.1** (5 bài) — đối chiếu lời giải; **VI.2** (10 bài) tự luyện, không kèm đáp án (có thể dùng để kiểm tra hoặc nộp BTL).

VI.1 Bài tập có lời giải**Bài 1 — Đường thẳng và nhãn trục**

Đề: Vẽ ($y = x^2$) trên $[-5, 5]$ với ít nhất 200 điểm; thêm tiêu đề mô tả nội dung, nhãn trục có đơn vị (nếu coi (x, y) là số thực), lưới nhẹ.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-5, 5, 200)
y = x**2

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(x, y, color="steelblue", linewidth=2, label=r"$y = x^2$")
ax.set_title("Parabol: bình phương theo x trên đoạn [-5, 5]")
ax.set_xlabel("x (đơn vị tùy chọn)")
ax.set_ylabel("y (đơn vị tùy chọn)")
ax.grid(True, linestyle="--", alpha=0.35)
ax.legend()
fig.tight_layout()
plt.show()
```

Bài 2 — Scatter theo góc phần tư (màu theo nhóm)

Đề: Tạo 120 điểm ngẫu nhiên trong $[-1, 1] \times [-1, 1]$. Tô màu theo góc phần tư (Q1–Q4); thêm legend.

```

import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
x = rng.uniform(-1, 1, 120)
y = rng.uniform(-1, 1, 120)

def quadrant_label(a, b):
    if a >= 0 and b >= 0:
        return "Q1 (+,+)"
    if a < 0 and b >= 0:
        return "Q2 (-,+)"
    if a < 0 and b < 0:
        return "Q3 (-,-)"
    return "Q4 (+,-)"

labels = np.array([quadrant_label(xi, yi) for xi, yi in zip(x, y)])
order = ["Q1 (+,+)", "Q2 (-,+)", "Q3 (-,-)", "Q4 (+,-)"]
colors = {"Q1 (+,+)": "C0", "Q2 (-,+)": "C1", "Q3 (-,-)": "C2", "Q4 (+,-)": "C3"}

fig, ax = plt.subplots(figsize=(7, 7))
for lab in order:
    m = labels == lab
    ax.scatter(x[m], y[m], s=28, alpha=0.75, c=colors[lab], label=lab,
edgecolors="k", linewidths=0.2)
ax.axhline(0, color="gray", linewidth=0.8)
ax.axvline(0, color="gray", linewidth=0.8)
ax.set_aspect("equal", adjustable="box")
ax.set_title("Scatter 120 điểm – màu theo góc phần tư")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.legend(loc="upper right", fontsize=8)
fig.tight_layout()
plt.show()

```

Bài 3 — Bar chart nhóm (so sánh hai năm)

Đề: 5 mặt hàng A–E; doanh số năm 1 và năm 2 (tự đặt số). Vẽ **grouped bar** có legend, nhãn trục rõ.

```

import numpy as np
import matplotlib.pyplot as plt

items = ["A", "B", "C", "D", "E"]
y1 = np.array([12, 19, 14, 22, 16])
y2 = np.array([15, 17, 18, 20, 21])

x = np.arange(len(items))
w = 0.35

```

```
fig, ax = plt.subplots(figsize=(9, 5))
ax.bar(x - w / 2, y1, width=w, label="Năm 1", color="steelblue",
edgecolor="black")
ax.bar(x + w / 2, y2, width=w, label="Năm 2", color="coral",
edgecolor="black")
ax.set_xticks(x)
ax.set_xticklabels(items)
ax.set_ylabel("Doanh số (đơn vị tùy chọn)")
ax.set_title("So sánh doanh số theo mặt hàng – hai năm")
ax.legend()
ax.grid(True, axis="y", linestyle="--", alpha=0.35)
fig.tight_layout()
plt.show()
```

Bài 4 — Lưới 2x2 subplot + `suptitle`

ĐỀ: Một figure: (0,0) sin, (0,1) cos, (1,0) histogram 800 mẫu chuẩn, (1,1) bar 3 cột. Dùng OOP; `fig.suptitle` tổng thể; `tight_layout` có chứa chỗ cho suptitle (`rect=[0,0,1,0.96]` hoặc `subplots_adjust`).

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2 * np.pi, 200)
rng = np.random.default_rng(0)

fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes[0, 0].plot(x, np.sin(x))
axes[0, 0].set_title("sin(x)")

axes[0, 1].plot(x, np.cos(x))
axes[0, 1].set_title("cos(x)")

axes[1, 0].hist(rng.normal(0, 1, 800), bins=35, edgecolor="black",
alpha=0.85)
axes[1, 0].set_title("Histogram – N(0,1)")

axes[1, 1].bar(["P1", "P2", "P3"], [4, 7, 5], color="seagreen",
edgecolor="black")
axes[1, 1].set_title("Bar 3 nhóm")

fig.suptitle("Bốn biểu đồ trong một figure", fontsize=14,
fontweight="bold")
fig.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

Bài 5 — GridSpec + lưu file PNG 300 dpi

Đề: Dùng `GridSpec`: một ô rộng phía trên (line plot), hai ô nhỏ hàng dưới (scatter + bar). Lưu `matplotlib_lab5.png` với `dpi=300`, `bbox_inches='tight'`.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec

x = np.linspace(0, 10, 150)
rng = np.random.default_rng(1)

fig = plt.figure(figsize=(10, 7), constrained_layout=False)
gs = GridSpec(2, 2, figure=fig, height_ratios=[2, 1], hspace=0.3,
wspace=0.25)

ax0 = fig.add_subplot(gs[0, :])
ax0.plot(x, np.sin(x), label="sin")
ax0.plot(x, np.cos(x), label="cos", alpha=0.85)
ax0.set_title("Hàng trên – full width")
ax0.legend()

ax1 = fig.add_subplot(gs[1, 0])
ax1.scatter(rng.random(40), rng.random(40), c=rng.random(40),
cmap="viridis", alpha=0.8)
ax1.set_title("Scatter")

ax2 = fig.add_subplot(gs[1, 1])
ax2.bar(["a", "b", "c"], [3, 7, 5], color="steelblue", edgecolor="black")
ax2.set_title("Bar")

fig.suptitle("Layout GridSpec – demo lab", fontsize=13)
plt.tight_layout(rect=[0, 0, 1, 0.95])
fig.savefig("matplotlib_lab5.png", dpi=300, bbox_inches="tight",
facecolor="white")
plt.show()
```

VI.2 Mười bài tự luyện

1. Vẽ **hai đường** trên cùng axes: ($y_1 = \sqrt{x}$), ($y_2 = \ln(x+1)$) với ($x \in [0.1, 5]$); dùng trục (y) log nếu cần để dễ so sánh (ghi chú trong caption/title).
2. **Histogram** hai tập dữ liệu chồng (alpha), cùng `bins`; thêm legend.
3. **Pie chart** 4 loại (≤ 7 slice), có `autopct`; thử `explode` một slice — nhận xét khi nào pie **không** nên dùng (liên hệ Phần III).
4. **Horizontal bar** 8 nhãn (tên dài giả); sắp xếp giảm dần theo giá trị.
5. **Error bar**: 10 điểm (x), (y) giả, `yerr` ngẫu nhiên dương; `capsize` rõ.
6. **Twin axes**: cột mưa (mm) + đường nhiệt ($^{\circ}\text{C}$) theo tháng 1–12 (dữ liệu giả); nhãn hai trục (y) khác màu.
7. **Annotation**: vẽ parabol, đánh dấu đỉnh bằng `annotate` + mũi tên.

8. **Style:** vẽ cùng một line plot dưới 3 `plt.style` khác nhau (3 figure hoặc 3 subplot); chọn style bạn thấy hợp báo cáo nhất.
9. **Pandas:** `DataFrame` có cột `time`, `a`, `b` — line plot hai cột `a,b` theo `time` bằng `.plot` hoặc matplotlib thuần.
10. **Báo cáo mini:** một figure **2×2** từ dữ liệu Titanic (hoặc CSV tự chọn): bar survival theo class, hist tuổi, scatter tuổi–fare, pie giới (hoặc thay pie bằng bar count nếu bạn phản đối pie) — **lưu PDF và PNG 300 dpi**.